



PROTOCOL ARCHITECTURE AND IMPLEMENTATION STATUS

PYRAX Network Technical Whitepaper

Version 4.0

A high-throughput Layer-1 with GhostDAG ordering,
TriStream consensus, and shielded value transfer.

Publication date June 30, 2026

Official website <https://pyraxnetwork.org>

Document status: technical draft.

Executive summary

PYRAX is a high-throughput Layer-1 protocol implemented in Rust. Its base ledger is a multi-parent DAG ordered by GhostDAG. TriStream combines two proof-of-work streams across four seal lanes with a proof-of-stake stream that provides BLS-aggregated finality. Production-mode transaction types default to shielded transfers; PYRAX Seed remains a transparent-only developer sandbox.

The architecture combines an L1 state commitment, three shared-backend L2 execution environments (EVM, WASM, and Cairo), and an L3 recursive validity-rollup design. L3 circuits and recursive verification are implemented, but the live bridge rejects all rollup proofs pending sequencer integration and external ZK audit. The network-wide target of at least 500,000 sustained transactions per second is not a demonstrated benchmark in this specification.

Peer discovery uses multiple libp2p mechanisms and a signed community seed artifact. Sphinx routing provides fixed-size onion packets. Cover-traffic emission and bandwidth-weighted routing remain unfinished; resistance to a general traffic-analysis adversary is not established.

NEURAX adds an escrow-settled compute marketplace whose core implementation resides in the protocol workspace; its proprietary repository and several services remain scaffolded or deferred. Crucible is a zero-developer-fee miner-manager scaffold, not a completed miner release. NOVA operates as an operations service with autonomy disabled by default and human approval for chain-service restarts.

The production supply cap is 50 billion PYRX: 37.5 billion premine and 12.5 billion mining emissions. Allocation arithmetic is consistent, but emission timing and AI-pool duration require reconciliation. Four chainspecs are specified: PYRAX Seed, PYRAX Forge, PYRAX Rise, and PYRAX One. Production mode is a consensus configuration, not evidence of mainnet readiness. Forge standup is recorded as in progress; Mainnet One readiness is not started.

Abstract

This whitepaper describes the architecture, protocol parameters, and implementation status of PYRAX. GhostDAG computes a canonical order over a multi-parent DAG; TriStream provides five seal lanes spanning proof of work and proof of stake, with stake-weighted BLS finality. Shielded transfers use note commitments, nullifiers, and a no-trusted-setup plonky2 proving backend. A composite state root commits account, privacy, escrow, staking, and governance state. EVM, WASM, and Cairo environments support synchronous cross-VM calls; recursive L3 validity-rollup settlement remains disabled. The document also specifies peer discovery, Sphinx transport, compute-market settlement, monetary parameters, and governance. Implemented code is distinguished from simulation, deferred integration, and external-audit dependencies; technical inconsistencies are explicitly identified.

Contents

Executive summary	i
Abstract	i
Definitions, abbreviations, and notation	v
1 Design goals	1
1.1 Performance target and measurement scope	1
2 System and protocol architecture	2
2.1 Transaction processing	3
2.2 Implementation organization	3
3 Consensus	4
3.1 DAG blocks and canonical ordering	4
3.2 TriStream and difficulty adjustment	4
3.3 Stake-weighted finality	5
3.4 Security scope	5
4 Privacy architecture	6
4.1 Shielded notes and public transaction data	6
4.2 Cryptographic construction and checks	6
4.3 Shielding boundary and conservation ambiguity	6
4.4 Metadata privacy and anonymous services	7
5 Execution environment	7
5.1 Shared-backend multi-VM execution	7
5.2 Fees, native functions, and transaction envelopes	8
5.3 L3 recursive validity rollup	9
6 Networking and peer discovery	9
7 State, storage, and proofs	11
8 NEURAX	11
8.1 Scope and implementation boundary	11
8.2 Job lifecycle and verification	12
8.3 Trust tiers and routing	13
9 Crucible	13
10 NOVA	14
11 Tokenomics	14
11.1 Token units and supply	14
11.2 Allocation and vesting policy	14
11.3 Mining emissions	16
11.4 Transaction-fee distribution	16
11.5 AI-compute funding and pricing	17
11.6 Staking rewards	17
12 Governance	18
12.1 On-chain proposal lifecycle	18
12.2 Licensing and operational independence	18
13 Network environments and deployment status	19

14 Wallet, applications, CLI, SDK, and RPC	20
14.1 Desktop and command-line applications	20
14.2 SDK and metrics	20
14.3 JSON-RPC surface	20
15 Cryptographic primitives	21
16 Security model and threat assumptions	22
16.1 Claimed invariants and adversaries	22
16.2 Threats and described mitigations	22
16.3 Trust-boundary interpretation	23
17 Known limitations and audit dependencies	24
17.1 External release gates	24
17.2 Specification gaps	25
18 Implementation-status matrix and roadmap	25
19 Conclusion	27
References	28
A Implementation module inventory	29
B Wire structures	30
C RPC inventory and interface review	31

List of Figures

1	PYRAX logical architecture and layer responsibilities	3
2	Logical transaction processing and finality	3
3	TriStream seal lanes, GhostDAG ordering, and BLS finality	5
4	Shielded transaction information boundary	7
5	Multi-VM dispatch and shared state	8
6	Discovery sources and peer transport	10
7	NEURAX compute-job lifecycle	12
8	Allocation of the 50B PYRX cap	15
9	Four chainspecs of one node binary	19
10	Wallet, ledger, compute, and operational trust boundaries	23

List of Tables

1	Definitions and notation	v
2	Design goals and mechanisms	1
3	Scaling mechanisms and implementation evidence	2
4	TriStream seal-lane parameters	4
5	Execution engines and limits	8
6	Transaction envelope tags	9
7	Peer transport and synchronization parameters	10
8	Mempool limits and scheduling	11
9	NEURAX verification ladder	12
10	NEURAX trust tiers	13
11	Token units and supply parameters	14
12	Supply allocation	15
13	Documented vesting policy	16
14	Consensus-frozen fee distribution	16
15	AI-compute economic parameters	17
16	Governable parameters and ceiling multipliers	18
17	Official network configurations	19
18	Cryptographic primitive inventory	21
19	Invariant families	22
20	Threats, mitigations, and qualifications	22
21	Known limitations	24
22	P0–P13 implementation roadmap	25
23	Component implementation and activation status	26
24	Cargo workspace inventory	29
25	RPC family inventory	31

Definitions, abbreviations, and notation

Table 1. Definitions and protocol notation.

Term	Meaning
DAG; GhostDAG	Directed acyclic graph; the k-cluster-based ordering mechanism.
Blue set; anticone	Selected compatible blocks; blocks unrelated by ancestry to a given block.
TriStream	Streams A and B (four PoW lanes in total), plus Stream C (PoS).
PoW; PoS; BFT	Proof of work; proof of stake; Byzantine fault tolerance.
DAA	Per-lane difficulty-adjustment algorithm.
L1; L2; L3	Base ledger; multi-VM execution layer; recursive validity-rollup layer.
ZK; zk-SNARK	Zero knowledge; succinct non-interactive argument of knowledge.
Anchor; nullifier	A note-tree root; a public spend marker checked for reuse.
AEAD; ECDH	Authenticated encryption with associated data; elliptic-curve Diffie–Hellman.
DHT; IPC; RPC	Distributed hash table; inter-process communication; remote procedure call.
TEE; CU	Trusted execution environment; Compute Unit (one reference-GPU-hour).
TGE; DAO	Token generation event; decentralized autonomous organization.
PYRAX; PYRX; ash	Network/token name; token ticker; base unit, 10^{-18} PYRX.
$k = 16$	GhostDAG blue-anticone bound.
h ; N ; $\parallel$	Block height; dispute trace length; byte-string concatenation.
W ; W_{att}	Total bonded stake weight; weight represented by finality attestations.
Production	Real-seal consensus mode; not a release-readiness certification.
Implemented	Completed code; external audit and deployment are separate statuses.

1 Design goals

PYRAX has five design goals. Whether each goal is achieved depends on the adversary, workload, and deployment configuration; the mechanisms below do not by themselves establish universal guarantees.

Table 2. Design goals, mechanisms, and qualifications.

Goal	Mechanism	Qualification
Concurrent block inclusion	Multi-parent DAG and GhostDAG ordering.	Throughput depends on ordering and network conditions.
Default value privacy	Shielded type; note commitments and nullifiers.	Seed is transparent-only; production release is audit-gated.
Decentralized discovery	mDNS, Kademlia, peer exchange, signed seed list.	Initial reachability and seed trust need specification.
Metadata privacy	Sphinx transport and fixed-size packets.	Cover emission and weighted routing are deferred.
Open verification	Apache-2.0 protocol, node, ZK circuits, CLI, and SDK.	Apps, wallet, and services are described as proprietary.

1.1 Performance target and measurement scope

The stated target is $\geq 500,000$ sustained transactions per second as a network-wide aggregate, with an unspecified “high L1 floor.” Simple value transfers define the headline metric; contract-call and shielded-transfer rates are to be reported separately. Performance reporting requires reproducible `pyrax-bench` measurements with workload, hardware, and bandwidth disclosed, but no measurement report is included in this specification.

Review R03. The throughput figure is a design target, not a measured result established by this document.

Table 3. Scaling mechanisms and implementation evidence. Unverified implementation states are identified explicitly.

Mechanism	Stated role	Status
DAG concurrency	Include concurrent blocks through GhostDAG ordering.	Implemented
Optimistic execution	Block-STM-style concurrency; deterministic re-execution on conflict.	Needs verification
L2/L3 batching	Move aggregate throughput above the L1 ledger.	L3 audit-gated
Recursive aggregation	Fold thousands of shielded proofs into approximately one verification per batch.	Needs verification for shielded batches
Availability sampling	Sample erasure-coded shares for light/mobile clients.	Needs verification
GPU proving	Reuse Stream-B hardware for shielded and rollup proofs.	Needs verification
Network efficiency	QUIC, compact/binary relay, transaction batching.	Binary relay implemented; other scope unverified

Archival, full, light, and mobile profiles are described as the means of retaining participation on consumer hardware. Resource budgets and conformance criteria remain to be defined.

2 System and protocol architecture

The protocol is organized as a Rust Cargo workspace with single-responsibility crates and a pinned `nightly-2026-06-10` toolchain. Its L1 combines GhostDAG ordering, TriStream seals, shielded state, a Merkle-Patricia account trie, and EIP-1559 fees. L2 exposes EVM, WASM, and Cairo execution over shared state. L3 recursively aggregates validity proofs for L1 settlement; activation of that settlement path is deferred.

The libp2p mesh and Sphinx transport are cross-cutting components, not additional settlement layers. Applications include the wallet, Inferno/Ember, CLI, anonymous file transfer, media streaming, and NEURAX. Figure 1 is a logical decomposition, not a claim that each component is independently deployed.

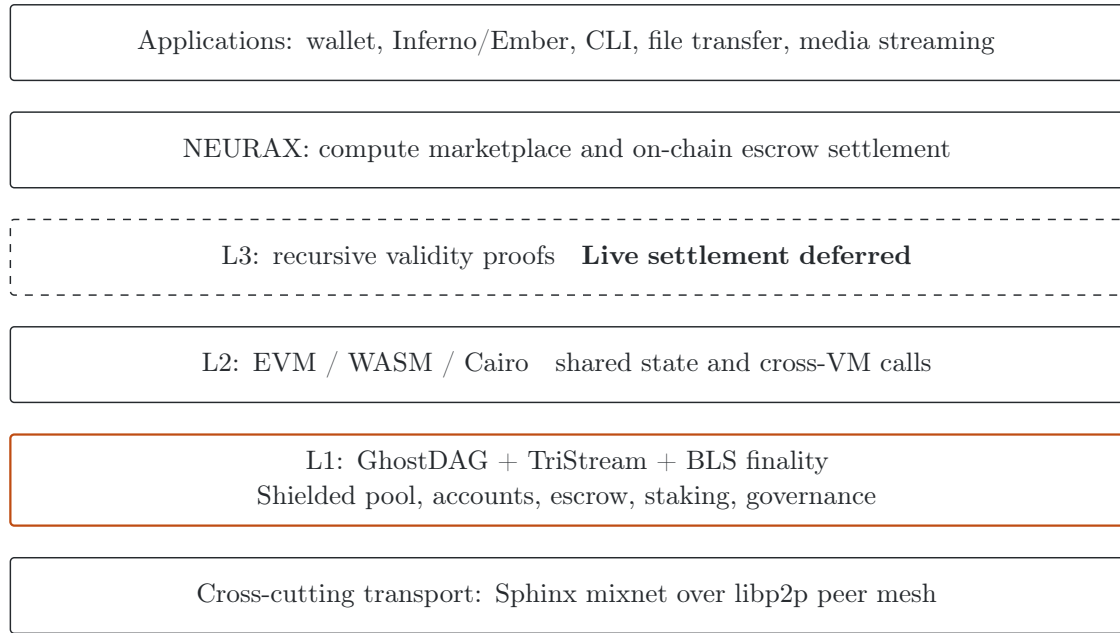


Figure 1. Logical architecture. Vertical placement groups responsibilities; it does not imply that NEURAX depends on L3. Dashed outlines indicate deferred activation.

2.1 Transaction processing

Submission enters transaction processing; the mempool organizes candidates, producers create DAG blocks, and GhostDAG supplies the canonical execution order used to compute state roots. Stream C validators attest to ordered blocks; a stake supermajority establishes finality. Figure 2 omits unsupported timing or concurrency guarantees.

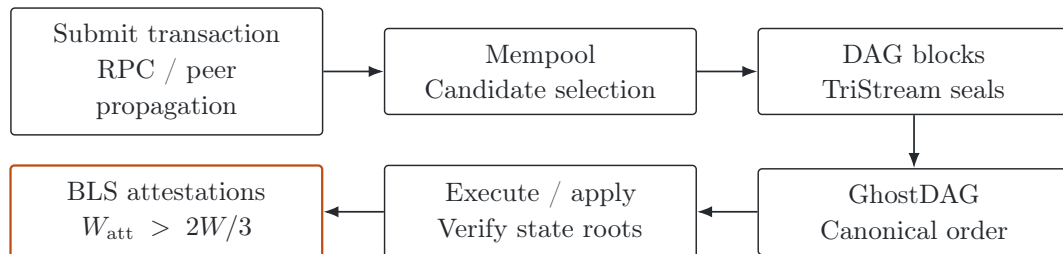


Figure 2. Logical transaction lifecycle. Arrows express processing dependencies, not latency guarantees or a complete validation algorithm.

2.2 Implementation organization

The principal responsibilities are consensus and ordering; cryptography and privacy; storage and state; peer and mixnet transport; multi-VM execution and rollups; compute settlement; and RPC, SDK, and node integration. Appendix A lists the crate inventory, including the distinction between the native `pyrax-contracts` crate and the separate, undeployed system-contract repository.

3 Consensus

3.1 DAG blocks and canonical ordering

A block may reference multiple current tips. The first entry, `parents[0]`, is the selected parent; following these links reaches genesis. Parent selection uses `blue_score`, `blue_work`, and hash, while the heaviest-blue-work tip determines fork choice. The block hash is BLAKE3 over canonical deterministic encoding, including parents and seal lane. Appendix B records the header fields.

GhostDAG uses a blue-set k -cluster rule with $k = 16$ [1, 2]. A mergeset candidate is blue when adding it preserves the blue-anticone bound. `blue_score` counts blue blocks in the past; `blue_work` is the saturating sum of blue-block difficulty. `order_dag` emits a topological mergeset order and then the block, with blue ahead of red and hash-based ties. This order determines execution and state roots.

`validate_block` recomputes score and work and enforces the selected-parent position. The implementation includes an ancestry index described as $O(1)$, differential tests against parent-DFS, and a bounded coloring cache replacing an $O(\text{chain}^3)$ merge-coloring pathology. These complexity bounds and tests require supporting artifacts.

Review R06. The score/work comparator and comparability of work across algorithms require a normative specification.

3.2 TriStream and difficulty adjustment

TriStream has three streams and five seal lanes (Table 4). Stream B uses kHeavyHash and Argon2id, not the KAWPOW, RandomX, or Cuckaroo superseded algorithm choices.

Table 4. TriStream seal lanes and hardware roles.

Stream	SealAlgo (tag)	Algorithm	Hardware
A	Blake3Pow (1)	BLAKE3(header) $\leq$ target	ASIC / specialized
A	Sha256dPow (2)	Double-SHA-256	ASIC
B	KHeavyHashPow (3)	64×64 matrix; one matrix multiplication + Keccak per nonce	Commodity GPU
B	Argon2idPow (4)	Argon2id; 8 MiB, $t = 1$, $p = 1$	Commodity CPU
C	PosBls (5)	BLS signature by slot-elected proposer	Staked validators

The producer chooses a lane, committed into the PoW preimage to prevent seal relabeling. Each lane’s DAA samples only same-lane ancestors along the selected-parent chain. The stated retarget rule is

$$\text{new_work} = \text{avg_work} \text{ clamp} \left(\frac{\text{expected}}{\text{actual}}, \frac{1}{4}, 4 \right). \quad (1)$$

Target comparison uses the high 128 bits of the hash. The sampling window and units of *expected* and *actual* are unspecified.

Seed uses `ConsensusMode::Simulated`: instant seals rotate A/B/C, gated by `produces_dev_blocks()`. Forge, Rise, and One specify `ConsensusMode::Production`; production chain IDs cannot use instant seals. The node auto-mines CPU-feasible Stream-A work and proposes Stream-C blocks; external miners supply Stream-B lanes using `pyrax_getWork` and `pyrax_submitWork`. The claim that a CPU cannot mine kHeavyHash “at difficulty” needs hardware and difficulty qualification.

The *entire subsidy goes to each block’s producer*. One-third reward shares for streams A/B/C are an aggregate expectation, conditional on each stream producing approximately one third of blocks; they are not a per-block three-way allocation.

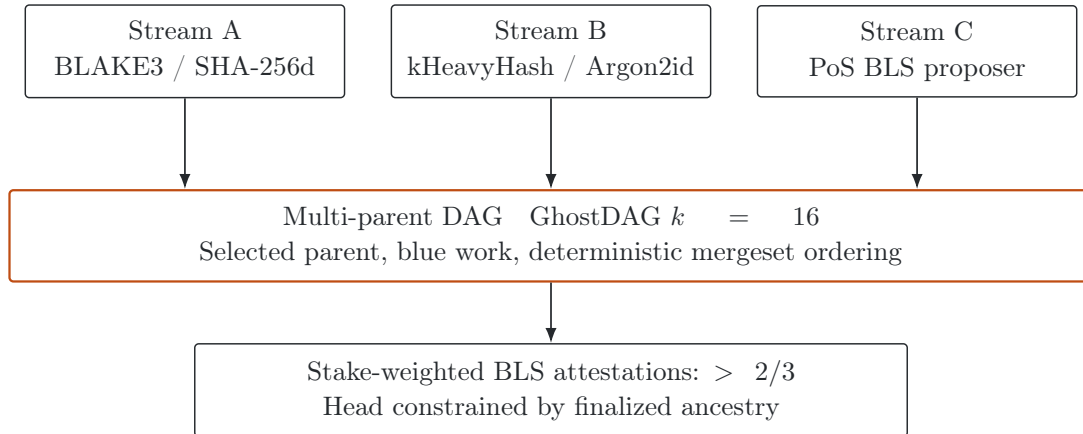


Figure 3. TriStream production, GhostDAG ordering, and BLS finality. Three streams contain five lanes; resource diversity alone does not establish a takeover threshold.

3.3 Stake-weighted finality

Validators bond at least 32 PYRX, with approximately seven days of unbonding. Votes use BLS12-381 min-pubkey signatures: 48-byte G1 public keys and 96-byte G2 signatures [3]. Registration requires proof of possession; aggregate verification checks subgroups and rejects infinity keys. Many signatures aggregate into one; signer-set encoding is unspecified.

$$W_{\text{att}} > \frac{2}{3}W. \quad (2)$$

A block is final when aggregate attestations satisfy Equation 2. The head must descend from the finalized hash: finality overrides work. Equivocation incurs a stated 5% burn and 10% reporter bounty; the basis of the bounty is unspecified.

3.4 Security scope

The consensus design posits that rewriting history requires majority Stream-A and Stream-B hashpower plus a staked supermajority in Stream C, treating these resources as uncorrelated. It also relies on withheld branches being colored red. These are security claims requiring formal justification, not established guarantees. The threat model separately assumes less than one third Byzantine stake for BFT safety. Their relationship, conflicting-finality handling, and the conditions under which withholding affects coloring require external review (Section 16).

4 Privacy architecture

4.1 Shielded notes and public transaction data

Production networks default to `Tx::Shielded`. Transparent transfers are explicit exceptions; Seed is transparent-only. The shielded pool uses an Orchard-style note/nullifier model [4, 5], not Orchard wire or circuit compatibility.

Shielded state contains an append-only commitment tree of depth 32 (2^{32} leaves) and a nullifier set. Roots serve as anchors. Creating a note appends its commitment; spending reveals its nullifier without identifying the spent commitment. Public fields are `chain_id`, `anchor`, nullifiers, commitments, signed 128-bit `value_balance`, ciphertexts, proof, and binding signature. Sender, receiver, and individual note amounts are not explicit fields. Appendix B retains the envelope.

Each shielded transaction burns `SHIELDED_FEE=100` base units. Repeated nullifiers are rejected against ledger state.

4.2 Cryptographic construction and checks

The default snark backend is plonky2 1.0.2 [6], using Poseidon over the Goldilocks field

$$p = 2^{64} - 2^{32} + 1. \quad (3)$$

Note commitments and position-independent nullifiers use domain separation:

$$cm = \text{Poseidon}(\text{DOM_CM}, \dots), \quad (4)$$

$$nf = \text{Poseidon}(\text{DOM_NF}, nk, cm). \quad (5)$$

Merkle nodes also use Poseidon; the transparent path uses BLAKE3. Notes use ChaCha20-Poly1305 with an X25519-ECDH shared secret and per-note nonce. Viewing keys separate viewing from spending authority.

The circuit is described as proving note membership at the anchor; correct nullifier derivation and spend authority; value conservation with hidden amounts range-checked to 60 bits; and well-formed outputs. Nullifier non-reuse is checked outside the proof. Ledger validation clamps `value_balance` to $\pm 2^{60}$, including for non-snark nodes. Verification runs on the apply path. `verify_pinned_circuits` rejects mismatched digests; this is an integrity control, not evidence of an external audit.

The recursive construction is intended to require no trusted setup. Halo provides background on recursive composition [7]; the implemented backend is plonky2, a distinct proof system.

4.3 Shielding boundary and conservation ambiguity

Transparent transfers use recoverable secp256k1 signatures with low-*s* enforcement. The transfer convention assigns positive `value_balance` to shielding (transparent value consumed, notes created), negative to deshielding, and zero to a fully shielded transfer. The conservation equation is

$$\text{inputs} = \text{outputs} + \text{value_balance}. \quad (6)$$

Review R12. The equation and sign convention conflict if inputs and outputs are shielded-note sums alone. This must be resolved before the equation serves as a normative rule.

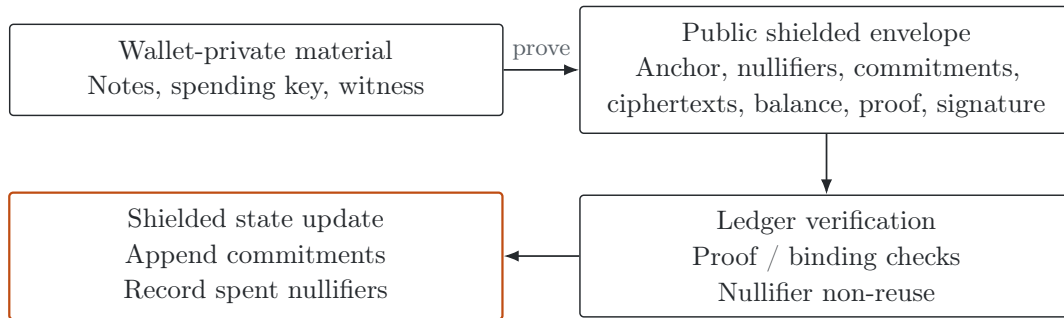


Figure 4. Shielded-transfer information boundary. Spending keys remain wallet-side. The unresolved balance-sign convention is not encoded in this diagram.

4.4 Metadata privacy and anonymous services

`pyrax-mixnet` provides Sphinx routing over Ristretto [8, 9], with `MAX_HOPS=5`, `PACKET_LEN=896` bytes, a 2^{20} replay window, constant-time MAC handling, and identity-point rejection. A relay removes one layer and forwards to the next hop. The broader assertion that it never learns source or destination requires endpoint and observer qualifications.

Cover-traffic policies and constructors exist, but the emitter loop and bandwidth-weighted route selection are deferred. Fixed packet lengths alone do not establish ISP resistance or absence of metadata leakage.

`pyrax-stream` uses BLAKE3-addressed chunks sized for an onion packet, with per-chunk capability-gated encryption keys and a content-addressed manifest. DHT advertisements support discovery; transfers are resumable. Live media is segmented, encrypted, and swarmed. A want/have pull protocol and signed, revocable relay directory support content-blind service. Paid-relay settlement remains a Phase-9 no-op stub; availability and origin independence are not independently demonstrated.

5 Execution environment

5.1 Shared-backend multi-VM execution

L1 is described as a lean settlement layer, with programmable execution in three anchored L2 environments. A shared `StateBackend` uses 20-byte addresses and 32-byte storage slots. Code magic bytes select the VM: `\0asm` selects WASM, `\0CAIRO` selects Cairo, and other code selects EVM.

Table 5. Execution engines, accounting rules, and limits.

VM / engine	Gas accounting	Determinism and limits
EVM / revm 22.0.1	Ledger-charged; revm uses <code>gas_price=0</code> / <code>disable_base_fee</code> ; BASEFEE remains real.	Journal/checkpoint transactionality; avoids double charging.
WASM / wasmtime 33.0.2	Fuel at 1:1.	NaN canonicalization; no threads or relaxed SIMD; stack 512 KiB; code 256 KiB; memory 16 MiB; logs 1 MiB.
Cairo / cairo-vm 2.5.0	Approximately one step per gas; 4M-step cap.	Non-canonical felt reverts; only <code>pyrax.*</code> hints.

Any VM may synchronously call another VM or a precompile. Gas forwarding follows the EIP-150 rule,

$$\text{forwarded} = \text{remaining} - \text{remaining}/64, \quad (7)$$

with revm handling EVM calls internally. `MAX_CALL_DEPTH=1024`; value moves exactly once, and inherited `STATICCALL` restrictions keep nested execution read-only.

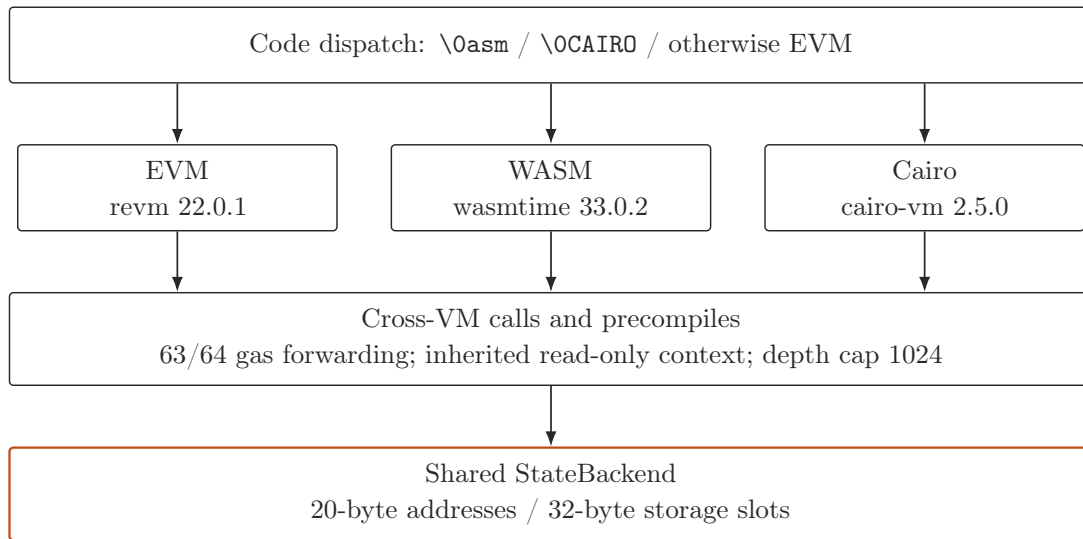


Figure 5. Multi-VM execution over shared state. The middle call layer represents synchronous cross-VM and precompile semantics, not an additional consensus layer.

5.2 Fees, native functions, and transaction envelopes

The fee-market parameters are `BLOCK_GAS_LIMIT=30,000,000`, a target of half that limit, base-fee movement of $\pm 12.5\%$ per block, and `MIN_BASE_FEE=1`. The frozen fee split is specified in Section 11; exact integer update rules require specification.

Table 6. Transaction envelope tags. All six are implemented.

Tag	Variant	Purpose
0	Shielded	Note/nullifier-based transfer.
1	Transparent	Recoverable secp256k1-signed transfer.
2	Ethereum	Raw EIP-2718 transaction; intended MetaMask/Foundry integration.
3	Escrow	NEURAX Lock, Refund, Release, Drip, Split.
4	Stake	Bond, Unbond, Withdraw, Slash.
5	Gov	Propose, Vote.

Precompiles are 0x0100 BRIDGE, 0x0101 BLAKE3, 0x0102 SHA256, 0x0103 KECCAK256, 0x0104 ECRECOVER, 0x0110 CHAIN_CONTEXT (chain ID, number, timestamp), and 0x0111 SHIELDED_VIEW (anchor).

Native-token, staking, bridge, ZK-verifier, and AI-compute-escrow functions are implemented in the core. The separate `pyrax-contracts` repository contains Phase-7 Solidity/Cairo system-contract stubs whose bodies revert; those contracts are not deployed. This repository must not be confused with the native core crate of the same name.

5.3 L3 recursive validity rollup

The L3 design batches transactions, proves each batch’s validity, recursively aggregates proofs two-to-one, and submits one proof for L1 settlement. It uses the same no-trusted-setup plonky2 stack as shielded transfers. The cryptographic core includes `pyrax-zk::rollup_circuit`, recursion, and an on-L1 verifier over a depth-32 sparse account tree (2^{32} accounts). The reported proving time for a production-depth aggregate is approximately 18 seconds; hardware and workload details remain to be documented.

The `pyrax-rollup` sequencer/execution code is described as built, but its prove/aggregate/post orchestration methods remain `unimplemented!()`. The live bridge deliberately uses a reject-all `DeferredVerifier`, preventing the rollup root from advancing until sequencer integration and external ZK audit. Implemented cryptographic components therefore do not establish operational L3 settlement.

6 Networking and peer discovery

`pyrax-p2p` uses `libp2p 0.55` [10]. The transport stack includes Noise, Yamux, QUIC/TCP, and WebRTC; their transport-specific composition requires clarification. Discovery combines LAN mDNS, Kademlia global crawl and rendezvous [11], peer exchange, and a signed community seed list that anyone may host. The seed list is a data artifact rather than a mandatory project-operated server.

This multi-input discovery design is described as “bootstrapless.” A configuration flag distinguishes discovery from operator behavior. Independence from any one operator is a design claim; cold-start reachability and adversarial seed distribution remain to be formally analyzed.

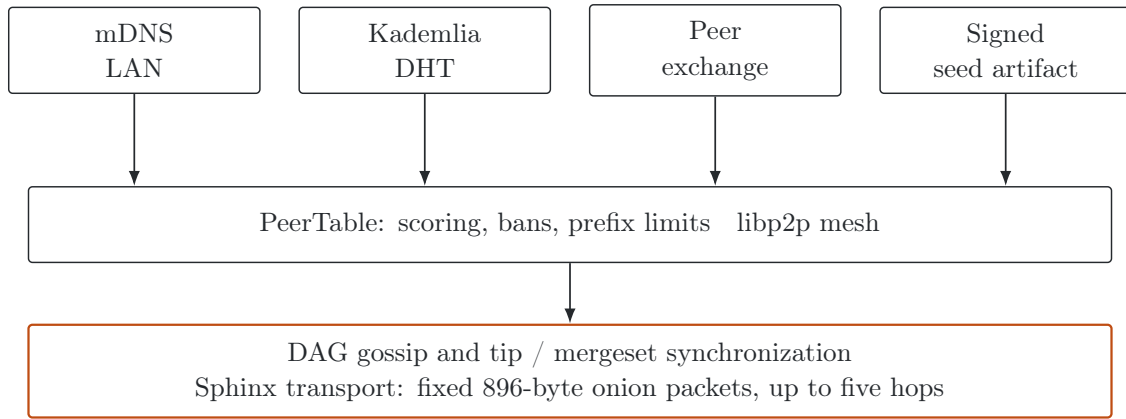


Figure 6. Multiple discovery inputs feed peer management and protocol transport. The signed artifact is not drawn as a central server. Cover-traffic emission remains deferred.

Table 7. Peer transport and synchronization parameters.

Parameter	Value or behavior
GossipSub	v1.1; Strict + Signed; six chain-scoped topics; peer scoring.
Message limit / heartbeat	8 MiB / 200 ms.
PeerTable	Live-wired; <code>max_peers=128</code> .
Network-prefix cap	Eight peers per IPv4 /24 or IPv6 /48.
Banning	Scores with escalating backoff; <code>PROTOCOL_VERSION</code> epoch deny-list.
DAG synchronization	Tip exchange and mergeset download.
Target corroboration	k-th-largest fresh peer tip; <code>SEAL_CORROBORATORS=2</code> .
Freshness	20 seconds.
No fresh target	A node at its own tip reports SYNCING, not a claimed 100%.
Wire format	Binary/CBOR; decoders are designed not to panic.

The “k-th-largest” peer statistic is distinct from GhostDAG’s $k = 16$ clustering parameter. The corroborator count is stated as two. The claim that minority liars cannot stall the seal gate needs a precise peer-adversary bound; the implementation currently gates synchronization on blue score alone.

7 State, storage, and proofs

Storage uses redb with LZ4 compression, described as a pure-Rust ACID key-value system with Blocks, State, Receipts, and Meta columns. RocksDB is not used by this storage design.

Account state uses a Merkle-Patricia trie with BLAKE3 nodes and secure-trie hashed keys. Each account leaf includes its actual storage root. Inclusion and exclusion proofs back `eth_getProof`; compatibility of PYRAX's BLAKE3 proofs with external Ethereum verifiers is not specified.

The composite root is

$$R = \text{BLAKE3}(D \| R_{\text{accounts}} \| R_{\text{notes}} \| A_{\text{nullifiers}} \| A_{\text{escrow}} \| C_{\text{staking}} \| C_{\text{gov}}), \quad (8)$$

where D is the literal domain string `PYRAX_STATE_ROOT_v3`. A and C denote the accumulator and commitment fields, respectively. The header's informal composite notation is not interpreted as bitwise XOR.

Fast-sync serves a root-verified `LedgerSnapshot` through `pyrax_snapshot`. The roadmap records finalized-snapshot fast-sync as implemented, with live-mesh auto-trigger integration remaining. Trusted finalized-root acquisition and snapshot authentication need specification.

Table 8. Mempool limits and scheduling rules.

Constraint	Value
Total entries	16,384
Per sender	256
Shielded entries	4,096
Ordering	Contiguous nonce runs; highest-fee head first.

8 NEURAX

8.1 Scope and implementation boundary

NEURAX is an on-chain-settled AI and compute marketplace intended to reuse GPU/CPU capacity associated with Stream B. Its proprietary `neurax-ai` repository is a scaffold. The marketplace implementation resides in the Apache core: `pyrax-job-market`, `pyrax-weights`, `pyrax-carriage`, `pyrax-local-runtime`, and `pyrax-model-registry`, with approximately 6.9k lines of code and 106 tests. These counts and test results require independent verification.

The design comprises tiered specialist experts for text, image, video, audio, ML-platform, and copilot functions behind an agentic router. It describes fine-tuning and distillation of open base models and use of established runtimes, not frontier pretraining. The dependency from proprietary components to core code is one-way; this architectural relationship does not establish completion of every service.

8.2 Job lifecycle and verification

Jobs have `JobKind=Inference|Training|Compute`; modality, tier, and parameters reside in a content-addressed specification at `Job.spec_hash`. The lifecycle is submit, escrow lock, redundant match, quorum execution, verification, and settlement or refund (Figure 7). A canonical `ComputeReceipt` supports settlement at a base rate of 8 PYRX/CU, metered in CU-milli. Escrow supports drip release and multi-party `release_split`.

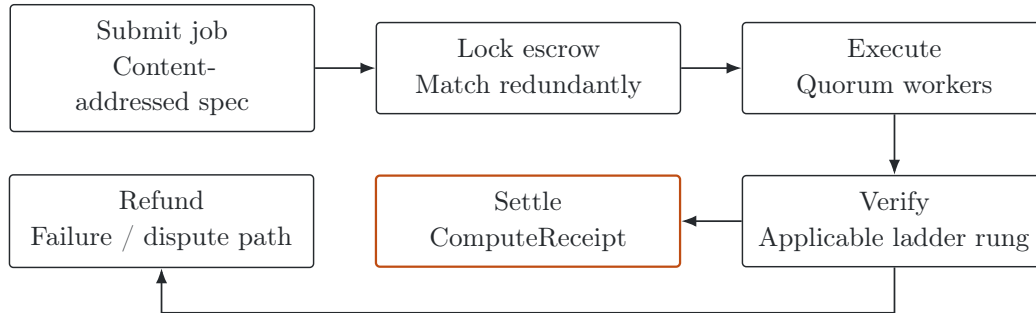


Figure 7. NEURAX job lifecycle. Settlement and refund are alternative outcomes; complete branch predicates and timeout rules remain to be specified.

Table 9. NEURAX verification ladder and limitations.

Rung	Mechanism	Qualification
1. Redundancy	Exact-hash quorum within a pinned <code>KernelClass</code> .	Cross-GPU inference is not assumed bit-reproducible.
2. Optimistic check	Fraud proofs with a challenge window.	Window and challenger incentives are unspecified.
3. Dispute	Merkle-trace bisection to one divergent step in $\lceil \log_2 N \rceil$ rounds.	The assertion that an honest defender always wins needs assumptions.
4. Attestation	TEE gating for Trusted jobs.	Real quote-chain verification is deferred despite the “all built” ladder claim.

GPU cohort pooling combines partial results with member-independent SHA-256; a Python sidecar is intended to reproduce this encoding byte-for-byte, with agreement between distinct honest cohorts requiring verification. Payout addresses derive from verified secp256k1 public keys rather than worker-supplied strings; per-shard CU clamps and canonical collision-safe spec encodings are reported. These controls require test evidence.

8.3 Trust tiers and routing

Table 10. NEURAX trust tiers. The tier ordering does not imply equivalent cryptographic assurance.

Tier	Definition
Open	Open-lineage weights; consumer GPUs permitted.
Secure/Sharded	Open models sharded across consumer GPUs; deterrence, not cryptographic proof.
Trusted	TEE-attested datacenter GPUs; proprietary weights and sealed jobs. Real quote-chain support is deferred.

A Trusted model requested on a consumer node routes to the network. `pyrax-local-runtime` is a deterministic, GPU-free router with RTX 3060 (12 GB) as the Baseline target; `decide_route` delegates to the network scheduler. `neurax-route` is shared by the desktop NEURAX tab and CLI. Equivalence of local and network routing decisions requires explicit test or proof evidence.”

Current performance expectations place consumer video generation on a minutes-long rather than real-time timescale, and WAN-sharded 100B+ inference at 1–5 tokens/s in batch processing. These estimates require workload-specific validation. The verification design assumes that ZK-ML cannot verify a 7–70B forward pass in 2026; this is a feasibility assumption requiring review, not a universal bound. WAN consumer training has no compute-integrity proof and instead relies on stake, reputation, and audit. Deferred items include real TEE quote chains, the NEURAX gateway/SDK, copilot, and 9.7 video runtime.

9 Crucible

`pyrax-crucible` is a design-complete miner-manager scaffold with a full build deferred until the L1 mainnet program passes. Its specified developer fee is zero. It points at the operator’s local node, with the operator’s own address as coinbase, no third-party pool, and no hidden routing. These are design properties, not evidence of a completed miner release.

The lane/hardware/backend matrix maps Stream-A BLAKE3 and SHA-256d to ASIC Stratum, Stream-B kHeavyHash to GPU, and Stream-B Argon2id to CPU. Stream C is staking-status only; Crucible does not mine a PoS lane. Solutions are intended to satisfy the node’s pure-Rust verifiers, but the deferred build prevents treating this as demonstrated end-to-end interoperability.

10 NOVA

`pyrax-nova` is implemented and operates at <https://status.pyraxnetwork.org>. Its operational cycle is patrol, deduplicate, investigate, assemble dossier, RAG-assess, dispatch, test-gated edge auto-repair, monitor, resolve, and learn. RAG denotes retrieval-augmented generation in this workflow; the retrieval corpus and assessment criteria are unspecified.

An uptime engine checks ecosystem surfaces, including a chain monitor that reports STALLED when height stops advancing, behind an SSRF guard. Faults route to the owning repository. An edge host runs a headless coding agent against an allow-listed checkout with a diff-level red-line file guard. The release workflow runs the full test suite, ships only on passing tests, and otherwise rolls back.

NOVA is advisory-first: autonomy is off by default; chain-service restarts always require human approval; a 15-minute warning precedes chain-affecting actions. Two red-line guards and an append-only action audit are described. The reasoning model runs locally on the operator’s GPU. Operational logs, guard policies, and rollout/rollback test evidence are required to validate these controls.

11 Tokenomics

Monetary parameters are defined in `TOKENOMICS.md` and implemented in `pyrax-consensus`, `pyrax-state`, and the native `pyrax-contracts` crate. Versioned constants and invariant-test evidence are required to establish correspondence with the parameters below. Arithmetic consistency alone does not establish code enforcement or completion of a distribution event.

11.1 Token units and supply

Table 11. Token parameters. Dollar figures are specified genesis terms, not current prices.

Field	Value
Name / ticker	PYRAX / PYRX
Decimals	18; 1 PYRX = 10^{18} base units (“ash”). EVM wei maps 1:1 to base units.
Maximum supply	50,000,000,000 PYRX; <code>assert_genesis_supply</code> on production networks. Seed alone waives the cap.
Composition	37.5B premine + 12.5B mined = 50B PYRX.
Genesis price	US\$0.0025/PYRX; approximately US\$125M fully diluted value at that stated price.

11.2 Allocation and vesting policy

Premine allocations are seeded into reserved credit-only system accounts (Table 12). The fee-funded PYRAX treasury is distinct from the DAO treasury.

Table 12. Allocation of the 50B PYRX cap. Amounts and percentages sum correctly; code enforcement is unverified.

Allocation	PYRX	Cap share	Account suffix
Genesis (public) distribution	25,000,000,000	50%	0x...0204
Ecosystem & liquidity	5,000,000,000	10%	0x...0205
AI-Compute & inference	4,000,000,000	8%	0x...0203
Team & advisors	2,500,000,000	5%	0x...0206
DAO treasury & reserve	1,000,000,000	2%	0x...0201
Premine subtotal	37,500,000,000	75%	—
Mining emissions	12,500,000,000	25%	Producer coinbase
Maximum supply	50,000,000,000	100%	—
PYRAX treasury (fee-funded)	0 at genesis	—	0x...0200

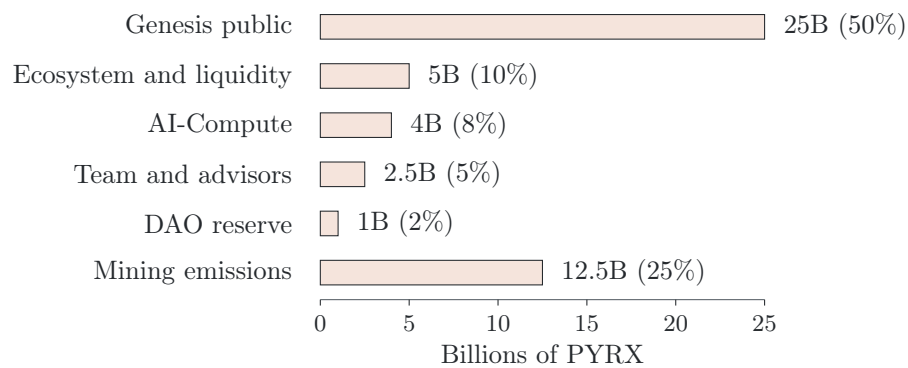


Figure 8. Allocation of the 50B PYRX cap. Bar lengths show token amounts; labels show cap shares. Allocation arithmetic is consistent; genesis execution and code enforcement require verification.

The described genesis event sells 20B PYRX at US\$0.0025 for US\$50,000,000, plus a 25% utility bonus of 5B PYRX. Participants therefore receive the entire 25B public-distribution pool. The genesis policy defines this bonus as network access/compute credits rather than investment return. Completion of the sale and receipt of proceeds remain unverified.

Table 13. Documented vesting policy; on-chain vesting contracts are deferred.

Pool	Policy
Genesis distribution	100% at TGE.
Mining	Approximately 26 years (timing requires review).
Ecosystem & liquidity	40% at TGE; 60% linearly over 24 months.
AI-Compute	Streamed over 48 months (in tension with budget duration).
Team & advisors	12-month cliff, then 36-month linear vesting.
DAO treasury & reserve	10% at TGE; remaining release schedule unspecified.

The reported initial circulation is approximately 27.1B PYRX (about 54%). The stated TGE amounts give $25 + 2 + 0.1 = 27.1$ billion, or 54.2% of the cap. This is policy arithmetic: credit-only accounts and deferred vesting contracts leave transfer authorization and actual circulation to be verified.

11.3 Mining emissions

INITIAL_BLOCK_SUBSIDY=300 PYRX. The subsidy is specified as

$$\text{subsidy}(h) = 300 \gg \left\lfloor \frac{h}{21,000,000} \right\rfloor, \quad (9)$$

clamped to the remaining 12.5B mining cap, with fees only after the cap. The right-shift notation is retained; whether it applies before or after conversion to base units is unspecified. The schedule specifies halvings every 21,000,000 blocks (approximately four years at six seconds), roughly 26 years to the cap, and roughly 80% mined in the first eight years. The genesis block mints no subsidy. A testnet-only ramp increases reward from 5% to 100% over 864,000 blocks.

The entire block subsidy is minted to its producer's coinbase. Approximate one-third shares for streams A/B/C are emergent from block-production frequencies, not a split of every block reward.

Review R24. The emission formula and time estimates require consistency verification. A normative base-unit schedule and cap-boundary test vectors are required.

11.4 Transaction-fee distribution

Table 14. Consensus-frozen fee distribution. Each row totals 100%.

Fee class	Burn	Producer	PYRAX treasury	DAO
Base fee	25%	0%	50%	25%
Priority tip	0%	70%	20%	10%

Transparent transfers use 21,000 intrinsic gas. The flat 100-base-unit shielded fee is burned. The PYRAX-treasury share is intended to fund AI-compute payouts when network revenue supports them.

11.5 AI-compute funding and pricing

One CU is one reference-GPU-hour (RTX-4090-class). The hardware multipliers used for CU accounting are: H100/A100 2.5–4.0×; RTX 4090 1.0×; RTX 3060/4060 0.35×; CPUs 0.04–0.15×. No measurement method or workload is attached.

Table 15. AI-compute economic parameters.

Parameter	Value
PYRX_PER_CU	8 PYRX/CU; CU-milli metering.
AI pool	4B PYRX.
Monthly pool budget	Approximately 70M PYRX (approximately 57 months).
Quality multiplier	0.8–1.2×.
Requester spot premium	Up to 2×.
Per-job pool drawdown cap	10,000 PYRX (1,250 CU at the base rate).
Revenue transition	Trailing-30-day coverage rising from 0 to 1 tapers the pool-funded payout share.
Pool retirement	Coverage ≥ 1.0 for three months replaces the pool with revenue; unspent funds return to the DAO treasury.

At the stated genesis price, 8 PYRX/CU corresponds to US\$0.02 per reference-GPU-hour. Early provider margin depends on token appreciation under this pricing model, creating an economic risk rather than an expected return. Revenue sources are job fees and the treasury gas share; escrow supports drip and split settlement.

Review R25. The 48-month vesting policy and approximately 57-month budget are separate parameters that require reconciliation. Their relationship remains unresolved.

11.6 Staking rewards

Validators earn the Stream-C emission share, the 70% producer tip on blocks they produce, and staking rewards. APR, commission, and delegation curves remain deferred to the emission workstream. Minimum stake is 32 PYRX; unbonding is approximately seven days; equivocation incurs a stated 5% slash and 10% reporter bounty.

12 Governance

12.1 On-chain proposal lifecycle

Tx: :Gov implements proposal, VOTING, automatic tally at the deadline, PASSED, and automatic enactment at an activation height. Any account may propose by locking 1,000 PYRX. The deposit is burned if quorum is not reached and refunded otherwise. Bonded validators vote with stake weighting; the latest vote wins.

$$W_{\text{vote}} \geq \frac{1}{3}W_{\text{bonded}}, \quad W_{\text{yes}} > \frac{2}{3}W_{\text{vote}}. \quad (10)$$

Exactly three parameters are governable (Table 16). Fee distribution, the 12.5B emission cap and halving schedule, and the governance rules themselves are immutable by governance.

Table 16. Governable parameters. The reference values for ceiling multipliers require confirmation.

Parameter	Ceiling
<code>block_gas_limit</code>	64×
<code>min_validator_stake</code>	1024×
<code>unbonding_period</code>	64×

The DAO treasury at `0x...0201` holds 1B PYRX at genesis, with 10% liquid under the documented policy, and receives 25% of base fees and 10% of tips. Treasury-spending governance is a later layer, not part of the three-parameter mechanism.

12.2 Licensing and operational independence

PYRAX uses an open-core model: the protocol, node, consensus, ZK circuits, CLI, SDK, and smart-contract interfaces are Apache-2.0 [12]; apps, wallet, NEURAX, and hosted services use `LicenseRef-PYRAX-Proprietary`. The licensing policy specifies SPDX identifiers at file starts and identifies PYRAX as a trademark. Repository licenses, ownership, and component boundaries require confirmation.

The stated decentralization policy excludes mandatory project-operated discovery infrastructure, uses a signed community seed artifact, and requires the explorer to respect shielded privacy. These policies do not establish achieved operator diversity, relay availability, or permissionless initial discovery under every failure model.

13 Network environments and deployment status

One binary ships four chainspecs with BLAKE3-pinned genesis hashes. Seed remains a sandbox; Forge, Rise, and One select real-seal Production consensus. Production identifies protocol behavior, not public availability or readiness. Genesis hashes, endpoint evidence, and a dated deployment inventory remain to be published.

Table 17. Official network names and configurations.

Network	Slug	Chain ID	Mode	Block	Role / qualification
PYRAX Seed	seed	881,109	Simulated	5 s	Permanent sandbox; approximately 1T play-money; cap waived; 1M welcome faucet.
PYRAX Forge	forge	710,823	Production	5 s	Closed public alpha; three seeded genesis validators, 0.1B PYRX each; standup in progress.
PYRAX Rise	rise	104,928	Production	6 s	Official public testnet; emission ramp; live deployment not established.
PYRAX One	one	563,821	Production	6 s	Mainnet; genesis ceremony pending; no faucet or development keys.

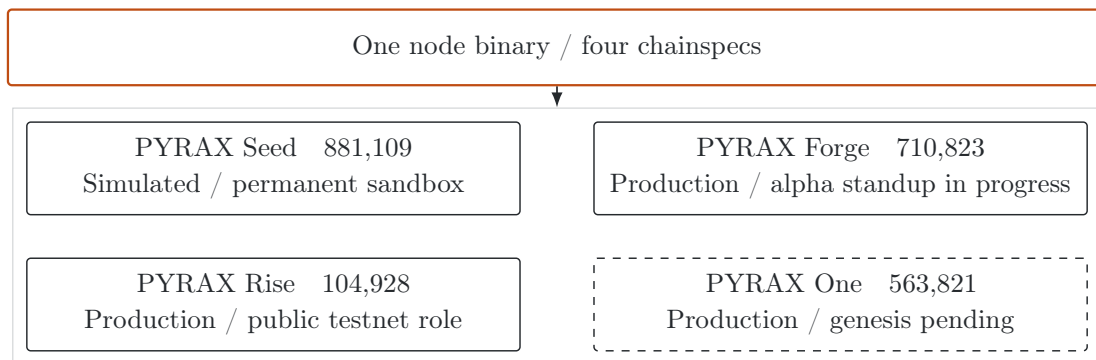


Figure 9. Four separate network configurations of one binary. The enclosing group denotes shared implementation, not a cross-network bridge or migration protocol. Dashed outline marks pending mainnet activation.

Seed shares the node binary but uses transparent-only transfers, instant seals, and a waived cap. Shared implementation does not mean identical network behavior. Forge, Rise, and One differ by chainspec, launch order, and audit gating.

Review R29. As-of deployment status is not equivalent to configured consensus mode. Forge is in progress; One's genesis is pending. Launch dates and live availability remain to be confirmed.

14 Wallet, applications, CLI, SDK, and RPC

14.1 Desktop and command-line applications

Inferno is the public operator Electron application: full node, stream mining, staking, and AI/compute participation, without seeding. Ember is the internal seed variant. The renderer is presentational; privileged actions cross a narrow typed IPC with `contextIsolation` enabled and `nodeIntegration` disabled. Desktop applications supervise the compiled node and three-stream miner through sidecars; the CLI embeds the node in-process.

The wallet description specifies script with $N = 2^{17}$, AES-256-GCM, OS-keychain sealing, and TOTP two-factor authentication, with fail-closed behavior. Keys remain in the main process; the shielded spending key is not sent to the node. Wallet-side proving uses public shielded chain data. Derivation uses BIP-44 coin type 7777 with per-network isolation. `pyrax-prover-wasm`, the browser/WASM prover, is a scaffold and audit-gated.

Review R31. Section 15 also identifies ChaCha20-Poly1305 for wallet-vault encryption. The implemented vault format must resolve this discrepancy.

The desktop OTA updater lacks in-app artifact-signature verification, relying on HTTPS and OS code signing. The CLI keystore KDF is not memory-hard and is weaker than the desktop wallet's stated script configuration. Signed releases in the roadmap do not resolve missing in-app verification.

14.2 SDK and metrics

`pyrax-sdk` is reported implemented, supporting offline build/sign/submit, typed methods, and a generic `call()` escape hatch. Prometheus metrics are exported by the node. The separate NEURAX gateway/SDK remains deferred and must not be confused with this protocol SDK.

14.3 JSON-RPC surface

The RPC surface comprises a stated 35 `eth_*` methods, six filter methods, `eth_subscribe` over WebSocket, `net_*/web3_*`, and 27 native `pyrax_*` methods. It is unclear whether the stated totals include filters and subscriptions. Appendix C lists the documented method families; a complete counted interface inventory remains to be verified.

Ethereum-compatible families cover chain identity and height; balances/nonces/code/storage; proofs; blocks, transactions and receipts; logs and filters; subscriptions for new heads, logs, and pending transactions; calls and gas estimation; fee queries; transaction submission; and synchronization. Log queries are capped at 10,000 blocks.

Native RPC adds honest synchronization status (height, target, `targetKnown`); shielded state and wallet-side proving data; snapshots; consensus/DAG inspection; peer/bandwidth management; external miner work; mixnet streams; and token-gated seed key generation and seed-list signing. Sensitive actions are not described as generally public.

`--rpc-public` enforces a strict read-only allowlist at the parsed JSON-RPC layer with DoS clamps. The read-only guarantee requires a full method, batch, and authorization specification.

15 Cryptographic primitives

The cryptographic implementation uses the following primitives in `pyrax-crypto` or the named privacy/mixnet components. The bibliography supplies background entries, not evidence that a composition or implementation is secure.

Table 18. Cryptographic primitives and uses.

Primitive	Use and checks
BLAKE3	Default hash; blocks, state nodes, file addressing, and Stream A [13].
SHA-256 / SHA-256d	Interoperability and Stream A; SHA-256 cohort-result encoding.
Keccak-256	EVM addressing and kHeavyHash components.
secp256k1	65-byte recoverable signatures; low- <i>s</i> transaction enforcement. <code>recover_evm</code> accepts high- <i>s</i> for ECRECOVER parity. Address: last 20 bytes of Keccak-256(public key).
Ed25519	Node/P2P and seed-list publisher identity.
BLS12-381 / blst	Min-pubkey aggregation; mandatory proofs of possession; subgroup and infinity-key validation.
ChaCha20-Poly1305	Note AEAD; also attributed to wallet vaults in this primitive inventory, conflicting with Section 14.
X25519	Note-encryption ECDH; low-order-point rejection.
Poseidon / Goldilocks	Shielded commitments, nullifiers, and note-tree nodes.
plonky2 1.0.2	Shielded and rollup circuits, recursive proofs, pinned digests.
Ristretto / Sphinx	Onion transport; identity-point rejection, MAC/replay checks.
Argon2id	Stream B memory-hard work; 8 MiB, $t = 1$, $p = 1$. The reference [14] has ambiguous attribution.
AES-256-GCM / scrypt	Wallet-vault construction described in Section 14; scrypt $N = 2^{17}$.

Low-*s* transaction validation and the ECRECOVER compatibility exception are distinct behaviors; the latter must not be generalized to transaction authorization. The Argon2id/RandomX bibliography entry requires correction and does not specify the current lane.

16 Security model and threat assumptions

16.1 Claimed invariants and adversaries

The security model includes an invariant catalog with enforcement sites and tests. Table 19 summarizes its identifiers and scope; the complete formal statements and supporting test artifacts remain to be documented. A catalog of tested assertions is not, by itself, a formal proof of protocol security.

Table 19. Protocol invariant families. Individual formal statements remain to be documented.

Family	Scope
M1–M4	Supply $\leq 50B$ on production networks; per-transaction conservation; frozen fee split; capped subsidy.
C1–C7	Unforgeable difficulty; execution-bound state root; finality overriding work; BLS supermajority; no honest equivocation; governed gas limit; cross-lane blue work.
Z1–Z4	Default real ZK verification; nullifier non-reuse; shielded conservation; L1 batch verification at depth 32.
G1–G4	Governance invariants; individual statements not included.
I1–I4	Replay protection; never-panic decoders; body integrity; authenticated senders.

Adversaries A1–A8 are malicious peer, Byzantine producer, minority staker/miner, stake-grinder, governance griever, resource-exhaustion attacker, privacy breaker, and key thief. The BFT safety assumption is less than one third Byzantine stake. This model still requires a complete network synchrony model, stake-snapshot rule, adaptive-corruption model, and adversarial resource-correlation model.

16.2 Threats and described mitigations

Table 20. Threat/defense mapping with explicit limitations.

Threat	Described defense and qualification
Consensus takeover	Resource diversity, blue-work ordering, BLS finality. The claimed simultaneous majority/supermajority threshold is not established by the stated assumptions.
Equivocation / nothing-at-stake	Stated 5% slash and 10% bounty; proof-of-possession-protected, subgroup-validated aggregates. Evidence and bounty basis are unspecified.
Shielded double spend	Reject reused nullifiers in the global set; depends on accepted state and correct validation.
Forged shielded value	plonky2 proof, binding signature, and pinned circuits. Balance-sign ambiguity and external audit remain unresolved.

Threat	Described defense and qualification (continued)
Graph / traffic analysis	Shielded fields and Sphinx packets. Cover emission is absent; no broad metadata-privacy guarantee is established.
Eclipse / spam / Sybil	Multiple discovery sources, GossipSub scores, prefix limits, escalating bans, and replay protection. Diversity limits are not identity scarcity.
Malicious compute worker	Sandbox execution and redundancy, fraud proofs, bisection, or TEE gating. Quote-chain verification remains deferred.
Signature malleability	Low- <i>s</i> authorization checks; high- <i>s</i> ECRECOVER compatibility is a separate exception.

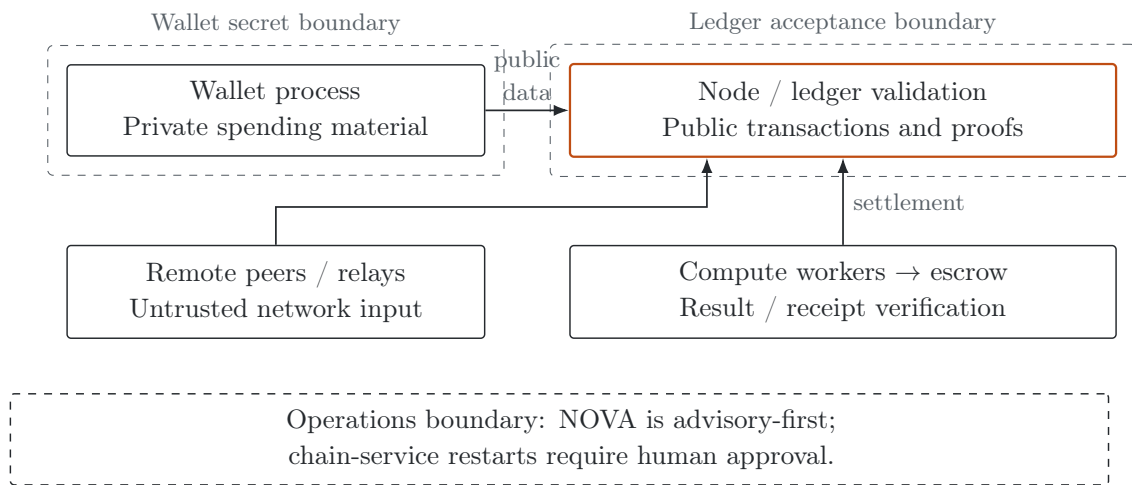


Figure 10. Security trust boundaries. Arrows cross validation boundaries, not guarantees of confidentiality or safety. Operational approval is distinct from consensus authorization.

16.3 Trust-boundary interpretation

The wallet retains private spending material; the node validates public transaction data. Remote peers provide untrusted network input. Compute workers submit results and receipts subject to the marketplace’s verification and escrow rules. NOVA’s operational automation is separate from consensus and cannot substitute for validation or release authorization. Figure 10 identifies these trust boundaries; isolation guarantees require separate validation.

17 Known limitations and audit dependencies

17.1 External release gates

Mainnet One is gated on external review of consensus design, ZK circuits, and bridge soundness. Shielded circuits will not ship to mainnet unaudited; GhostDAG parameters and finality rules also require external review. An incentivized PYRAX Rise testnet and a bug bounty are to precede genesis. Auditor details, audit reports, testnet outcomes, and bounty-launch evidence are not included in this specification; completion of these prerequisites remains unverified.

Deployment is a deliberate human action; the system does not self-deploy. External audit is a release gate, but sequencer integration and other implementation/readiness work remain separate blockers.

Table 21. Known implementation and deployment limitations.

Area	Limitation and boundary
L3 integration	Sequencer prove/aggregate/post orchestration is unimplemented; the live bridge rejects all proofs and cannot advance its root.
Mixnet	Cover emission and bandwidth-weighted route selection are not yet built.
Synchronization	Blue-score-only gating; live-mesh fast-sync auto-trigger remains a residual.
System contracts	Separate Solidity/Cairo repository is an undeployed reverting scaffold. Native core functions are distinct.
Browser proving	<code>pyrax-prover-wasm</code> is scaffolded and audit-gated.
Crucible	Design-complete scaffold; full build deferred until the L1 mainnet program passes.
Desktop updates	No in-app artifact-signature verification; HTTPS and OS code signing only.
CLI keystore	Non-memory-hard KDF; weaker than desktop scrypt.
NEURAX	Real TEE quote chains, gateway/SDK, copilot, and 9.7 video runtime deferred; WAN consumer training lacks a compute-integrity proof.
Relay payments	Paid-relay settlement is a Phase-9 no-op stub.
Vesting / DAO spending	On-chain vesting and treasury-spending governance are later phases.
Validator key management	Remote signer remains outstanding in P11.
Performance	No reproducible 500k-TPS benchmark, workload disclosure, or independent proving/compute performance evidence.

17.2 Specification gaps

The unresolved items are not confined to missing implementations. Supply timing, shielded conservation, cross-lane work, finality conditions, wallet encryption, peer bootstrapping, and RPC compatibility require normative detail or reconciliation. These gaps must be resolved before the affected rules can serve as a complete normative specification.

18 Implementation-status matrix and roadmap

Implementation status is recorded as of June 30, 2026. *Implemented* denotes completed code, not external audit or live deployment. *Simulated* denotes sandbox behavior; *Planned* denotes deferred work; *Audit-gated* denotes a release dependency. *Needs verification* identifies unresolved evidence or scope. An “in progress” phase does not establish completed testing or deployment.

Table 22. Real-Consensus to Mainnet program, preserving P0–P13.

Phase	Status	Scope and qualification
P0	Implemented	Cluster harness, 50B supply invariant, Production consensus spine.
P1	Implemented	Merkle-Patricia state with storage in root.
P2	Implemented	State proofs and <code>eth_getProof</code> .
P3	Implemented; residual planned	Finalized-snapshot fast-sync; live-mesh auto-trigger remains.
P4	Implemented	BLS PoS finality and staking economy.
P5	Implemented	Five-lane TriStream and per-lane DAA.
P6	Implemented	P2P hardening and binary codec; Sybil/eclipse-resistance claims require independent validation.
P7	Implemented core; audit-gated	L3 verifier/prover done; sequencer go-live deferred; 500k TPS remains a target.
P8	Implemented; mainnet audit-gated	ZK privacy default-on for production networks.
P9	Implemented	Live-network CI, fuzzing, property/invariant tests; supporting artifacts require publication.
P10	Implemented	SDK and Prometheus metrics.
P11	Implemented in part; remainder planned	On-chain governance, double-sign guard, and signed releases done; remote signer remains.
P12	Needs verification	Forge closed-alpha standup: In progress. This does not identify completed deployment or test results.
P13	Planned; audit-gated	Mainnet One readiness and external audit package: Not started.

Table 23. Component-level status separating code, simulation, and activation.

Component	Status	Boundary
PYRAX Seed	Simulated	Permanent instant-seal sandbox; transparent-only.
GhostDAG / TriStream / BLS	Implemented	Implemented and tested; mainnet audit still required.
State, proofs, three VMs	Implemented	Cross-VM calls, precompiles, six envelopes, Merkle-Patricia state.
Shielded prover	Implemented; audit-gated	Default-on, pinned digests; unaudited mainnet release prohibited.
L3	Implemented core; audit-gated	Real circuits/recursion; disabled settlement orchestration and bridge.
Peer mesh / Sphinx	Implemented in part	Scoring and fixed packets built; cover emission and weighted routing planned.
File / media services	Implemented in part	Content-addressed transfer and streaming; paid settlement stubbed.
NEURAX core	Implemented; audit-gated track	Job lifecycle, escrow, verification ladder, pooling, router; real TEE quote chains deferred.
NEURAX services	Planned / scaffold	Proprietary repository, gateway/SDK, copilot, video runtime incomplete.
Crucible	Planned / scaffold	Full build deferred.
Desktop, wallet, CLI, SDK	Implemented	Browser prover remains scaffolded; updater and KDF limitations apply.
NOVA	Implemented	Reported live; autonomy off by default.
Tunnel, sync worker, directory	Implemented	Architecture and detailed scope remain to be documented.
Five websites / deploy bundle	Implemented	Complete website identities and deployment evidence remain to be documented.
Optimistic execution, DA sampling, GPU proving	Needs verification	Scaling claims do not establish implementation state.

NEURAX's 9.x program and Crucible follow their own tracks behind separate audit gates. L3 sequencer go-live and external L1 audit are principal remaining items before Mainnet One. Dates and a complete readiness checklist remain to be defined.

19 Conclusion

PYRAX's design combines GhostDAG ordering, five-lane TriStream production, BLS stake finality, and shielded note-based transfers in a high-throughput Layer-1. Multi-VM execution, recursive rollup components, peer/mixnet transport, compute settlement, and operator tooling extend that base.

The implementation has concrete readiness boundaries: Seed is simulated; L3 settlement is disabled; mixnet cover emission, several compute services, browser proving, and Crucible remain incomplete; and mainnet requires external audit and human-controlled deployment. The 500,000-TPS target is not a demonstrated performance result.

A release-quality protocol specification requires reconciliation of the flagged conservation, emission, consensus, privacy, and wallet details; versioned evidence for implementation claims; and complete references and audit artifacts. These open questions constrain the security, performance, and readiness claims that can be made for the protocol.

References

- [1] Y. Sompolinsky, S. Wyborski, and A. Zohar. PHANTOM and GhostDAG: A Scalable Generalization of Nakamoto Consensus. Publication metadata requires completion; verification required.
- [2] The Kaspas project. GhostDAG / k-cluster blue-set ordering; kHeavyHash. Topic description, not an identified publication. Verification required.
- [3] D. Boneh, M. Drijvers, and G. Neven. Compact Multi-Signatures—BLS aggregation and proofs of possession. Exact work and implementation correspondence require verification.
- [4] E. Ben-Sasson, A. Chiesa, et al. Zerocash—the note-commitment / nullifier model. Exact title, authors, and metadata require verification.
- [5] The Electric Coin Company / Zcash. Orchard shielded protocol. Protocol revision and exact reference require verification.
- [6] Polygon Zero. Plonky2: Fast Recursive Arguments with PLONK and FRI. Exact reference and correspondence to version 1.0.2 require verification.
- [7] S. Bowe, J. Grigg, and D. Hopwood. Recursive Proof Composition without a Trusted Setup (Halo). Metadata requires verification; background, not the stated backend.
- [8] G. Danezis and I. Goldberg. Sphinx: A Compact and Provably Secure Mix Format. Publication metadata requires completion; verification required.
- [9] Nym / Loopix. Anonymous communication via stratified mixnets with cover traffic. This entry conflates systems; identify intended works separately. Verification required.
- [10] Protocol Labs. libp2p. Exact documentation and release references require verification.
- [11] P. Maymounkov and D. Mazières. Kademlia. Full title and metadata require completion; verification required.
- [12] Apache Software Foundation. Apache License, Version 2.0. Application to project components requires verification.
- [13] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn. BLAKE3. Author attribution and specification revision require verification.
- [14] tevador. Argon2id / RandomX memory-hard proof-of-work. This entry conflates algorithms and attribution; verify the intended references. RandomX is not a current lane.

A Implementation module inventory

Table 24. Cargo workspace modules and responsibilities.

Crate / family	Responsibility
<code>pyrax-crypto</code>	Hashes, signatures, BLS proofs of possession, AEAD, X25519, privacy primitives.
<code>pyrax-primitives</code>	Multi-parent block types, six-variant Tx envelope, EIP-1559 parameters, SealAlgo.
<code>pyrax-codec</code> , <code>pyrax-proto</code>	Canonical binary codec and protobuf bridge; decoders are designed not to panic.
<code>pyrax-dag</code>	DAG store, k-cluster coloring, blue work, per-lane DAA, reachability.
<code>pyrax-consensus</code>	TriStream/GhostDAG orchestration, DevSealer, rewards/emissions, ConsensusMode.
<code>pyrax-zk</code>	Shielded circuits, L3 validity/recursion, pinned circuit digests.
<code>pyrax-privacy</code>	Poseidon note tree, nullifiers, viewing keys, note encryption.
<code>pyrax-mixnet</code>	Sphinx transport over Ristretto; five-hop maximum, 896-byte packets.
<code>pyrax-stream</code>	E2E file/media transport, chunk DAG, want/have pull, signed relay directory.
<code>pyrax-p2p</code>	libp2p mesh, DAG gossip, PeerTable eclipse/ban controls.
<code>pyrax-storage</code> , <code>pyrax-state</code> , <code>pyrax-mempool</code>	redb/LZ4 storage, Merkle-Patricia state/proofs, transaction pool.
<code>pyrax-vm-evm</code> , <code>pyrax-vm-wasm</code> , <code>pyrax-vm-cairo</code> ; <code>pyrax-contracts</code> (crate)	Multi-VM runner, precompiles, native protocol functions; distinct from undeployed system-contract repository.
<code>pyrax-rollup</code>	Sequencer/execution; cryptographic core in <code>pyrax-zk</code> ; prove/aggregate/post wiring deferred.
<code>pyrax-job-market</code> , <code>pyrax-weights</code> , <code>pyrax-carriage</code> , <code>pyrax-local-runtime</code> , <code>pyrax-model-registry</code>	Compute market, model CDN, activation channel, local-GPU router, model registry.
<code>pyrax-rpc</code> , <code>pyrax-sdk</code> , <code>pyrax-node</code>	JSON-RPC, typed SDK, node binary.

B Wire structures

These are descriptive field inventories, not compilable Rust definitions or a complete binary encoding. Hash, Timestamp, Stream, proof, and binding-signature encodings remain to be fully specified.

Listing 1. DAG header field inventory.

```
DagBlockHeader {
    parents: Vec<Hash>
    blue_score: u64
    blue_work: u128
    daa_score: u64
    state_root: Hash
    transactions_root, receipts_root: Hash
    timestamp: Timestamp
    stream: Stream
    seal_algo: SealAlgo
    seal: Vec<u8>
}
```

`parents[0]` is the selected parent. Stream is A, B, or C; SealAlgo identifies one of the five lanes. The state root commits the account, note, nullifier, escrow, staking, and governance components in Equation 8. Blue score/work and selected-parent selection are recomputed during validation.

Listing 2. Shielded transaction field inventory.

```
ShieldedTransaction {
    chain_id, anchor,
    nullifiers: Vec<Hash>,
    commitments: Vec<Hash>,
    value_balance: i128,
    ciphertexts: Vec<Vec<u8>>,
    proof, binding_sig
}
```

Fully shielded transfers use `value_balance=0`. The shielding/deshielding signs conflict with the conservation equation if inputs and outputs denote shielded-note sums (Review R12). Encoding and authorization details require further specification.

C RPC inventory and interface review

This inventory lists RPC method names and wildcard families. The stated totals of 35 Ethereum and 27 native methods require verification against a complete API inventory.

Table 25. RPC method families.

Namespace / role	Named methods or families
eth_*	chainId, blockNumber, getBalance, getTransactionCount, getCode, identity/state, getStorageAt, getProof.
eth_* history	getBlockBy*, getBlockReceipts, getTransactionBy*/Receipt, getLogs (10k-block cap).
eth_* execution/fees	call, estimateGas, gasPrice, maxPriorityFeePerGas, feeHistory, sendRawTransaction, syncing.
Filters / WebSocket	Six filter methods (individual names not enumerated); eth_subscribe: newHeads, logs, newPendingTransactions.
Other compatibility	net_*, web3_*; exact lists remain to be enumerated.
pyrax_* state/sync	blockNumber, syncStatus (height, target, targetKnown), getBalance, nonce, getBlock, noteState, shieldedChainData, snapshot.
pyrax_* consensus	consensusInfo, dagTips, dagRecent, getWork, submitWork, sendTransaction.
pyrax_* peers	peerCount, peers, nodeInfo, bandwidth, dial, dropPeer.
pyrax_* services	Mixnet stream* calls; token-gated seedKeygen, seedListSign; subscribeNewHeads.

Ethereum compatibility applies to the documented method families and requires conformance testing, particularly for BLAKE3 state proofs and strict public-mode mutation restrictions.